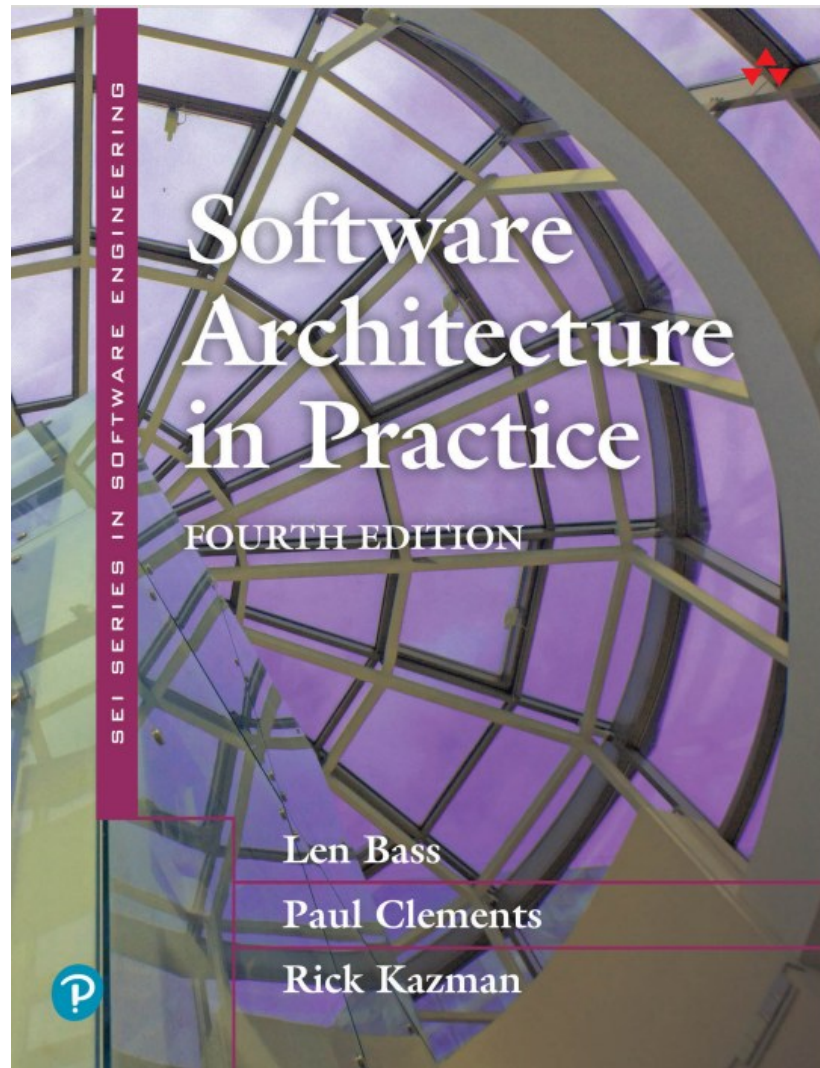




University Of Isfahan

Software Architecture in Practice



Dr. Mohammadreza Sharbaf



PART 3

Architectural Solutions



CHAPTER 15

Software Interfaces



“

NASA lost its \$125-million Mars Climate Orbiter because spacecraft engineers failed to convert from English to metric measurements when exchanging vital data before the craft was launched. . . .

A navigation team at [NASA] used the metric system of millimeters and meters in its calculations, while [the company that] designed and built the spacecraft provided crucial acceleration data in the English system of inches, feet and pounds. . . .

In a sense, the spacecraft was lost in translation.

”

—Robert Lee Hotz, “Mars Probe Lost Due to Simple Math Error,” Los Angeles Times, October 1, 1999

Chapter Outline



- Introduction
- Interface Concepts
 - Multiple Interfaces
 - Resources
 - Operations, Events, and Properties
 - Interface Evolution
- Designing an Interface
 - Interface Scope
 - Interaction Styles
 - Representation and Structure of Exchanged Data
- Documenting the Interface
- Summary

Introduction



- In Chapter 1, we defined architecture in terms of elements and their relationships. In this chapter, we focus on one type of relationship.
- An interface is a boundary across which elements interact. Interfaces control access to element's internals.
- Interfaces have large impact on a system's modifiability, usability, testability, performance, integrability, and more.
- For a given element's interface, there can be one or more implementations, each of which might have different performance, scalability, or availability guarantees.

Introduction



- Interface terminology:
 - Element actors: users, or systems with which it interacts.
 - Environment of the element: The collection of actors with which an element interacts.
 - Resources: These constructs that provide points of direct interaction with an element (such as procedure calls and message passing)
- Three points are implied by the discussion thus far:
 - All elements have interfaces;
 - otherwise, what is the point of the element's existence?
 - Interfaces are two-way
 - Interface is not just about what an element provides, it is also about what it uses.
 - An element can interact with more than one actor through the same interface.
 - For example, web servers often restrict the number of HTTP connections that can be open simultaneously.

Interface Concepts: Multiple Interfaces



- It is possible to split a single interface into multiple interfaces. Each of these has a related logical purpose, and serves a different class of actors.
 - A specific class of actor might require only a subset of the functionality available; this functionality can be provided by one of the interfaces.
 - Conversely, the provider of an element may want to grant actors different access rights, such as read or write, or to implement a security policy.
- For example, an element might expose its functionality through its main interface and give access to debugging or performance monitoring data or administrative functions via separate interfaces.

Resources



- Don't confuse resources with data or computational resources. Here we mean constructs that let elements interact, Like calling an API.
- Resources have syntax and semantics:
- Resource syntax: The syntax is the resource's signature. The signature includes the name of the resource, the names and data types of arguments, if any, and so forth
- Resource semantics: What is the result of invoking this resource?
 - Changing data: Maybe it sets a value or updates the database.
 - Changing its internal state: It might remember something or behave differently next time.
 - Visible real-world effects: In embedded systems, this might mean turning on a light or display screen.
 - Sending events or messages: It might trigger a signal or notify other parts of the system.

Operations, Events, and Properties



- When a system provides an interface, it offers resources that others can use to interact with it as. It also should provide description about resource. These resources are mainly of three types:
- Operations:
 - These are like *functions or methods* you call.
 - You call them to send some data and possibly get a result back.
 - They might fail, so the interface should explain about it.
- Events:
 - These are usually asynchronous, meaning they happen on their own, not because someone directly called them.
 - Incoming events are what the element *receives*.
 - Outgoing events are what the element *sends out* to notify others.

Operations, Events, and Properties



- Properties:

- In addition to the data transferred via operations and events, an important aspect of interfaces is metadata, such as access rights, units of measure, or formatting assumptions.
- Another name for this interface metadata is properties.
- For example:
 - Access rights (who is allowed to use the resource)
 - Units of measure (e.g., is a temperature in Celsius or Fahrenheit?)

Interface Evolution

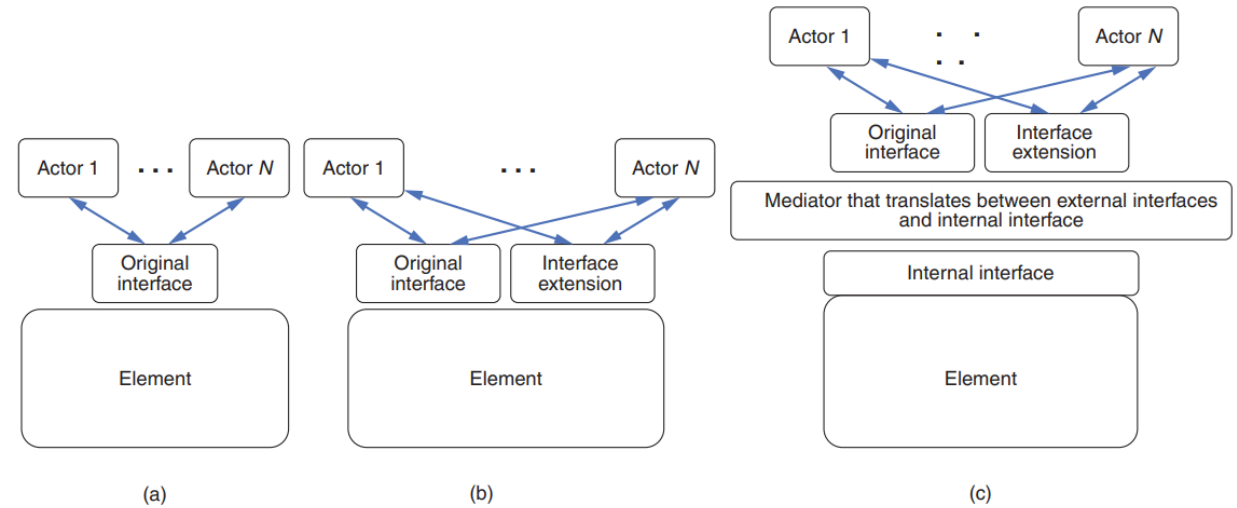


- As software grows and changes over time, the interfaces may also need to change.
- The software itself is free to evolve without impact to the elements that use this interface as long as the interface itself does not change.
- An interface, however, is a contract between an element and its actors so software interfaces should be changed with care. Here are three techniques change an interface:
 - **Deprecation**: Deprecation means removing an interface.
 - When deprecating an interface, the actors of the element should be noticed.
 - **Versioning**: This means you keep the old version and add a new one.
 - You can later remove the old version when it's no longer needed.
 - **Extension**: add new features to the interface without changing the old ones.
 - There are two ways to do this: Compatible Extension and Incompatible Extension

Interface Evolution



- Figure (a) shows the original interface.
- If the extension does not contain any incompatibilities with the original interface, then the element can implement the external interface directly, as shown in Figure (b).



- If the extension introduces some incompatibilities, it's necessary to add a mediator to translate between the external interface and the internal interface, as shown in Figure (c).

Designing an Interface



- Here we discuss some design principles for interfaces:
 - **Resources stability**: Once a resource is publicly available, it cannot be changed or removed. Otherwise, it would break users' systems.
 - **Principle of Least Surprise**: Make sure the interface behaves the way the users expect it to. For example, name resources correctly.
 - **Small interfaces principle**: If two elements need to interact, have them exchange as little information as possible.
 - **Uniform access principle**: Avoid leaking implementation details through the interface.
 - **Don't repeat yourself principle**: Interfaces should offer a set of composable primitives as opposed to many redundant ways to achieve the same goal.
 - **Consistency**: Consistency is an important aspect of designing clear interfaces. As an architect, you should establish and follow conventions on how resources are named, how API parameters are ordered, and how errors should be handled.

Designing an Interface



- A successful interaction with an interface requires agreement on the following aspects:
 1. Interface scope
 2. Interaction style
 3. Representation and structure of the exchanged data
 4. Error handling
- Each of these constitutes an important aspect of designing an interface. We'll cover each in turn.

Interface Scope



- The scope defines which resources are available to the actors. As a designer, you can limit access to specific resources or actors.
- A common way to control access is by using a **gateway** element. A gateway translates actor requests into requests for the target element's resources. This is useful when:
 - The granularity of resources provided by an element may be different than an actor needs. A gateway can translate between elements and actors.
 - The actor needs access to only a subset of resources.
 - The details of resources (like number, protocol, location) change over time. The gateway can provide a stable interface.

Interface Scope

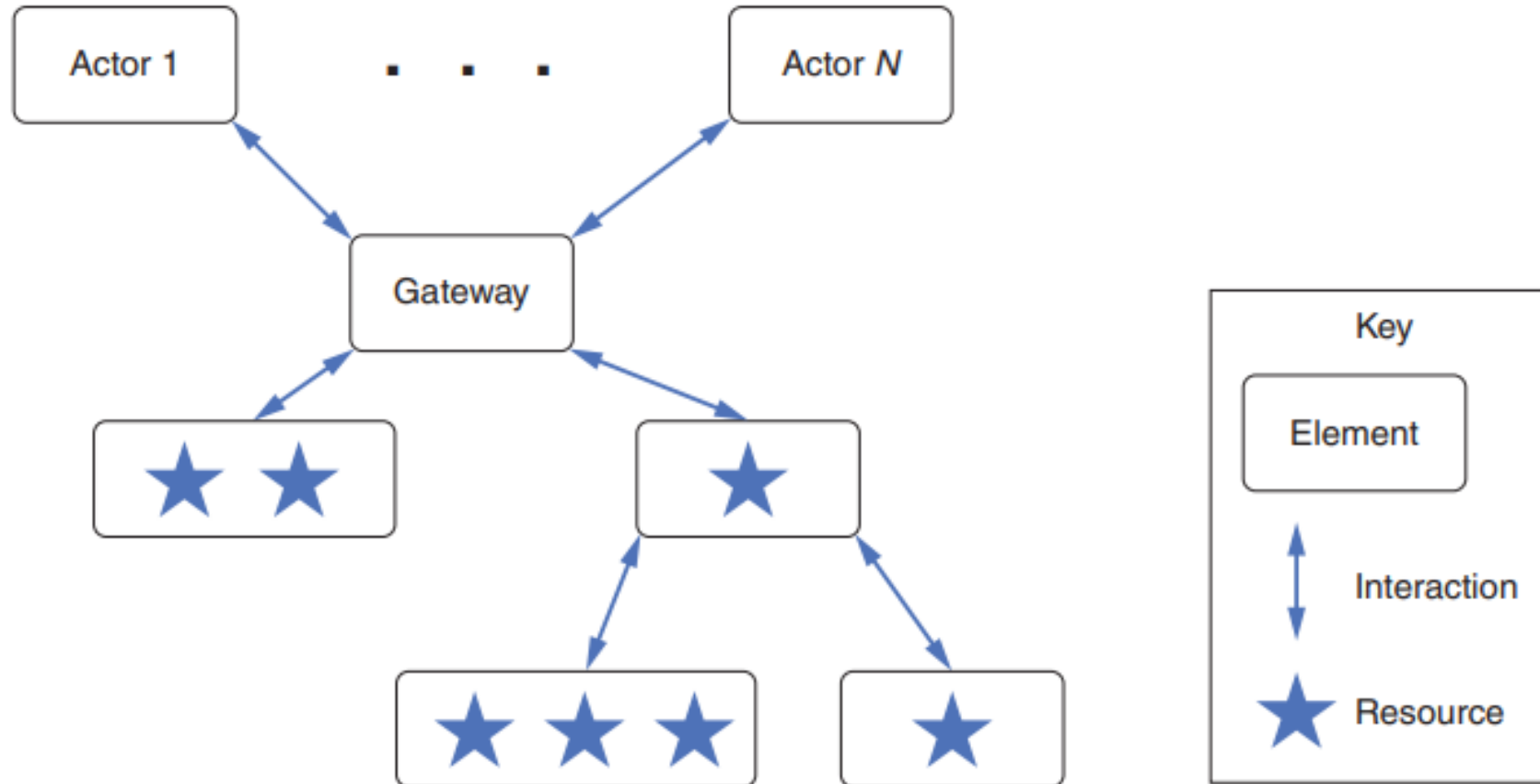


Figure of a gateway that provides access to a variety of different resources.

Interaction Styles



- Interfaces are meant to be connected together. There are many ways for such interaction depending on will be co-located or remotely deployed. For example:
 - **co-located elements**: may provide access to large quantities of data via local shared memory buffers.
 - **Elements that are expected to be available at the same time**: can use synchronous calls.
 - **Elements deployed in an unreliable distributed environment**: should use asynchronous interactions
- Many different interaction styles exist, but we will focus on two of the most widely used: **RPC** and **REST**
- **Remote Procedure Call (RPC)**
 - RPC is modeled on procedure calls in imperative languages, except that the called procedure is located elsewhere on a network.
 - The early versions of this protocol were synchronous, with the parameters of the message being sent as text. The most recent RPC version, called gRPC, transfers parameters in binary, is asynchronous, and supports authentication, cancellation, timeouts and many more.

Interaction Styles



- **Representational State Transfer (REST)**

- REST is a protocol for web services.
- REST comprises a set of six constraints on the interactions between elements:
 - **Uniform interface:** All interactions use the same form (typically HTTP). Resources on the providing side of the interface are specified via URIs .Naming conventions should be consistent . in general, the principle of least surprise should be followed.
 - **Client-server:** The actors are clients and the resource providers are servers.
 - **Stateless:** All client-server interactions are stateless. That is, the client should not assume that the server has retained any information about the client's last request. In consequence, interactions such as authorization are encoded into a token and the token is passed with each request.
 - **Cacheable:** Caching is applied to resources when applicable.
 - **Tiered system architecture:** The “server” can be broken into multiple independent elements, which may be deployed independently. For example, the business logic and the database can be deployed independently.
 - **Code on demand:** (optional) It is possible for the server to provide code to the client to be executed. JavaScript is an example.

Representation and Structure of Exchanged Data



- Interfaces gives you a chance to hide the internal details of how data is stored inside a program.
 - Instead of directly sharing these internal data structures, we convert them into a form that can be easily sent over a network or used by another program written in a different language.
 - This process of converting the internal format into a shareable one is called: [serialization](#), [marshaling](#) or [translation](#).
- we focus on the selection of a data interchange format or representation for sending information over a network. This decision is based on the following concerns:
 - *Expressiveness*: data representation power
 - *Interoperability*: Does the representation match what its actors expect and know how to parse?
 - *Performance*: bandwidth usage, complexity of parsing and marshaling the representation.
 - *Implicit coupling*: shared assumptions that could lead to errors when decoding messages.
 - *Transparency*: possibility of intercepting messages and observe the content. (a double-edged sword)

Representation and Structure of Exchanged Data



- Here are the most common data representation styles:
 - **EXtensible Markup Language (XML)**
 - XML is a meta-language that allows you to define your own markup for structuring data using tags and attributes.
 - It supports validation through schemas but is verbose and costly in terms of parsing and performance.
 - **JavaScript Object Notation (JSON)**
 - JSON uses nested name/value pairs and arrays to represent data in a lightweight, readable format.
 - It's less verbose than XML, faster to parse, and widely used in web applications.
 - **Protocol Buffers**
 - Protocol Buffers are a compact, binary serialization format developed by Google with high performance and low bandwidth usage.
 - They require schema definitions and language-specific compilers, and are commonly used with gRPC.

Documenting the Interface



- The interface documentation indicates what other developers need to know about an interface to use it in combination with other elements
- Also, different people need to know different kinds of information about the interface. You may have to include separate sections in the interface documentation that accommodate different stakeholders of the interface. keep the following stakeholder roles in mind:
 - *Developer of the element*
 - *Maintainer* (A special kind of developer who makes assigned changes to the element and its interface)
 - *Developer of an element using the interface*
 - *Systems integrator and tester:*
 - *Analyst*
 - *Architect looking for assets to reuse in a new system*

Summary



- Architectural elements have interfaces, which are boundaries over which elements interact with each other.
- Interface design is an architectural duty, because compatible interfaces allow architectures with many elements to do something productive and useful together. A primary use of an interface is to encapsulate an element's implementation, so that this implementation may change without affecting other elements.
- Interfaces have operations, events, and properties; these are the parts of an interface that the architect can design.
- Some of these issues can be addressed by standardized means. For example, data exchange can use mechanisms such as XML, JSON, or Protocol Buffers. All software evolves, including interfaces. Three techniques that can be used to change an interface are deprecation, versioning, and extension. The interface documentation indicates what other developers need to know about an interface to use it in combination with other elements. Documenting an interface involves deciding which element operations, events,